JavaScript Interview Cheat Sheet

JavaScript Interview Cheat Sheet (2 YOE)

1. Shallow Copy vs Deep Copy

Shallow copy creates a new object but only copies references to nested objects/arrays — the nested part still points to the same memory as the original.

```
const original = { a: 1, nested: { b: 2 } };
const shallow = { ...original };

shallow.nested.b = 99;
console.log(original.nested.b); // 99 — original is modified!
```

Deep copy recursively clones every level, so the copy is fully independent.

```
const original = { a: 1, nested: { b: 2 } };
const deep = structuredClone(original);

deep.nested.b = 99;
console.log(original.nested.b); // 2 — original unchanged
```

Feature	Shallow Copy	Deep Copy
Nested objects	Shares references	New instances at every level
Performance	Fast	Slower (recursive)
Independence	Partial	Full
How to create	`{...obj}`, `Object.assign({}, obj)`, `arr.slice()`	`structuredClone(obj)` (recommended), `JSON.parse(JSON.stringify(obj))`

Limitations of `JSON.parse(JSON.stringify())`: silently drops/breaks `undefined`, functions, `Date` (converts to string), `Map`/`Set`, `Symbol`, and fails on circular references. `structuredClone()` handles all of these correctly and is the modern default.

When to use which: shallow copy when only top-level fields change; deep copy when nested objects must be fully independent (e.g. cloning state you'll mutate before comparing to the original).

2. var, let, and const

	`var`	`let`	`const`
Scope	Function-scoped	Block-scoped	Block-scoped
Redeclare	Yes	No (error)	No (error)
Reassign	Yes	Yes	No
Hoisting	Hoisted, initialized `undefined`	Hoisted, TDZ	Hoisted, TDZ
Must initialize at declaration	No	No	Yes

```
function test() {
  if (true) { var x = 10; }
  console.log(x); // 10 — var leaks out of the block
}

if (true) { let y = 10; }
console.log(y); // ReferenceError — let is block-scoped

const user = { name: "John" };
user.name = "David"; // allowed — only the reference is constant
user = {};           // TypeError — cannot reassign the binding itself
```

Interview answer: "`var` is function-scoped and can be redeclared/reassigned. `let` is block-scoped, allows reassignment but not redeclaration. `const` is block-scoped and can't be reassigned after initialization — though object/array contents it points to can still be mutated. In modern JS I default to `const`, use `let` when a value needs to change, and avoid `var`."

---

3. Hoisting

Hoisting is JavaScript moving declarations to the top of their scope before execution.

Declaration	Hoisted	Initial value	Access before declaration
`var`	Yes	`undefined`	Returns `undefined`
`let`	Yes	Not initialized	`ReferenceError` (TDZ)
`const`	Yes	Not initialized	`ReferenceError` (TDZ)
Function declaration	Fully hoisted	Function object	Callable
Function expression (`var greet = function(){}`)	Variable only	`undefined`	`TypeError` (not a function yet)
Arrow function (`let` / `const`)	Variable only	TDZ	`ReferenceError`

---

4. Temporal Dead Zone (TDZ)

The TDZ is the span between entering a scope and the line where a `let` / `const` variable is actually declared. Accessing the variable during this window throws a `ReferenceError`. It exists to stop code from depending on a variable before its intended lifecycle begins.

```
{
  console.log(x); // ReferenceError — x is in the TDZ
  let x = 5;
}
```

---

5. Closures

A closure is formed when an inner function retains access to variables from its outer function's scope, even after the outer function has returned.

```
function outer() {
  let count = 0;
  function inner() {
    count++;
    console.log(count);
  }
  return inner;
}

const counter = outer();
counter(); // 1
counter(); // 2
counter(); // 3
```

Why it works: every function keeps a reference to its **lexical environment** (local vars, parent-scope vars, globals) at creation time. Normally local variables are destroyed when a function returns, but since `inner` still references `count`, JS keeps it alive.

Uses: data privacy (private variables), callbacks, event handlers, maintaining state between calls (like the counter above), memoization.

Downside: closures that retain references to large objects no longer needed can prevent garbage collection, increasing memory usage — avoid holding onto references you don't need.

Classic interview task — counter via closure:

```
function createCounter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}
const counter = createCounter();
console.log(counter(), counter(), counter()); // 1 2 3
```

---

6. The Event Loop

JavaScript is single-threaded but handles async operations (timers, network calls, DOM events) without blocking, via the event loop.

Components 1. **Call Stack** — where synchronous code executes 2. **Web APIs / Node APIs** — handle `setTimeout`, `fetch`, DOM events outside the stack 3. **Microtask Queue** — `Promise.then/catch/finally`, `queueMicrotask()` (higher priority) 4. **Macrotask / Callback Queue** — `setTimeout`, `setInterval`, DOM events 5. **Event Loop** — repeatedly checks: is the call stack empty? If so, run **all** pending microtasks, then run **one** macrotask, then repeat.

```
Call Stack empty?
   ├── Yes → drain Microtask Queue fully → run 1 Macrotask → repeat
   └── No  → keep executing
```

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
// Output: 1, 4, 3, 2
// Sync code first, then microtasks (Promise), then macrotasks (setTimeout)
```

Ways to handle async operations: callbacks → Promises → `async/await` (preferred, most readable).

---

7. `==` vs `===`

- `==` (loose equality) — compares after type coercion
- `===` (strict equality) — compares value **and** type, no coercion

```
console.log(5 == "5");  // true  — "5" coerced to 5
console.log(5 === "5"); // false — different types
```

Prefer `===` by default to avoid surprises from implicit coercion.

---

8. Promises

A Promise represents the eventual result (or failure) of an async operation. It replaced deeply nested callbacks (“callback hell”) with cleaner chaining.

States: `pending` → `fulfilled` (resolved) or `rejected`.

```
fetch("/users")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.log(err))
  .finally(() => console.log("done either way"));
```

Method	Behavior
`Promise.all(promises)`	Resolves when **all** succeed; rejects immediately if **any** rejects
`Promise.allSettled(promises)`	Waits for all to finish; returns each result's status (`fulfilled` / `rejected`) regardless of outcome
`Promise.race(promises)` *(added)*	Settles as soon as the **first** promise settles (fulfilled or rejected)
`Promise.any(promises)` *(added)*	Resolves as soon as the **first** promise fulfills; rejects only if **all** reject

```
// Promise.race — useful for timeouts
const withTimeout = Promise.race([
  fetch("/data"),
  new Promise((_, reject) => setTimeout(() => reject("Timeout"), 5000))
]);

// Promise.any — useful for "first available" fallbacks (e.g. multiple mirrors/CDNs)
Promise.any([fetch("/mirror1"), fetch("/mirror2")])
  .then(res => console.log("First success:", res));
```

9. async/await

Syntactic sugar over Promises — doesn't replace them, makes them easier to read.

- An `async` function always returns a Promise
- `await` pauses execution *inside that function* until the Promise settles — it does **not** block the whole thread or the event loop; other code continues running

```
async function getUser(id) {
  try {
    const res = await fetch(`/users/${id}`);
    const user = await res.json();
    return user;
  } catch (err) {
    console.error("Failed to fetch user:", err);
    throw err;
  }
}
```

10. null vs undefined

	`undefined`	`null`
Meaning	Variable declared, not assigned	Intentionally set to "no value"
`typeof`	`"undefined"`	`"object"` (historical bug)
Equality	`null == undefined` → `true`	`null === undefined` → `false`

```
let a;
console.log(a);            // undefined
let user = null;
console.log(user);         // null
console.log(typeof null);  // "object" — well-known JS bug, null is still a primitive
```

`undefined` shows up when a variable isn't assigned, a missing object property is accessed, or a function has no `return`. `null` is something a developer explicitly assigns.

11. Higher-Order Functions (HOF)

A function that takes another function as an argument, returns a function, or both.

```
function greet(name) { return `Hello, ${name}`; }
function processUser(callback) { console.log(callback("Sapna")); }
processUser(greet);
```

Built-in examples: `map()`, `filter()`, `reduce()`, `forEach()`, `setTimeout()`.

12. Scope

Scope Type	Declared where	Accessible where
Global	Outside all functions	Everywhere
Function	Inside a function	Only inside that function
Block	Inside `{}` with `let` / `const`	Only inside that block

Block scope prevents variables from leaking outside their block, reducing bugs. Typical use: global scope for config/constants, function scope for helper vars inside utilities, block scope (`let` / `const`) inside loops and conditionals to avoid accidental leakage.

13. Function Declaration vs Function Expression

```
// Declaration — fully hoisted, callable before its line
greet(); // works
function greet() { console.log("Hello"); }

// Expression — stored in a variable, NOT fully hoisted
greet2(); // ReferenceError (TDZ, since const)
const greet2 = function () { console.log("Hello"); };
```

Function expressions and arrow functions are the norm in modern React/Node code for callbacks, handlers, and async logic.

14. Arrow Functions vs Regular Functions

Regular functions have their own `this`, determined by how they're called. Arrow functions have **no own** `this` — they inherit it from the enclosing lexical scope.

```
const person = {
  name: "Sapna",
  greet: function () { console.log(this.name); }
};
person.greet(); // "Sapna" — this === person

const person2 = {
  name: "Sapna",
  greet: () => { console.log(this.name); }
};
person2.greet(); // undefined — this comes from outer scope, not person2
```

15. The `this` Keyword

`this` refers to the object executing the current function; its value depends on **how** the function is called, not where it's defined (except for arrow functions).

Context	Value of `this`
Global scope (browser)	`window`
Object method	The object calling the method
Regular function, non-strict	Global object (`window`)
Regular function, strict mode	`undefined`
Arrow function	Inherits from surrounding lexical scope
Constructor (`new Fn()`)	The newly created object
`call()` / `apply()` / `bind()`	Whatever object you explicitly pass

```
const person = {
  name: "Sapna",
  greet() {
    const print = () => console.log(this.name); // arrow inherits `this` from greet()
    print();
  }
};
person.greet(); // "Sapna"
```

`call`, `apply`, `bind` — explicitly set `this`:

```
function greet() { console.log(this.name); }
const person = { name: "Sapna" };

greet.call(person);              // "Sapna" — args passed individually
greet.apply(person);             // "Sapna" — args passed as an array
const bound = greet.bind(person); // returns a NEW function with `this` permanently set
bound();                          // "Sapna"
```

One-liner to remember: regular functions get `this` from how they're called; arrow functions inherit `this` from where they're defined.

---

16. Memoization

Caches the result of an expensive function call so repeated calls with the same input return the cached value instead of recomputing.

```
function memoize(fn) {
  const cache = new Map();
  return function (...args) {
    const key = JSON.stringify(args);
    if (cache.has(key)) {
      return cache.get(key);
    }
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
}

function slowSquare(n) {
  for (let i = 0; i < 1e8; i++) {} // simulate expensive work
  return n * n;
}

const fastSquare = memoize(slowSquare);
fastSquare(5); // slow — computes and caches
fastSquare(5); // fast — returned from cache
```

Used for expensive pure computations, and conceptually related to `useMemo` / `useCallback` in React.

---

17. Generators and Iterators

An **iterator** is an object that lets you traverse a collection one value at a time via a `.next()` method returning `{ value, done }`. A **generator** is a special function (`function*`) that automatically produces an iterator, using `yield` to pause and resume execution.

```
function* numberGenerator() {
  yield 1;
  yield 2;
  yield 3;
}

const gen = numberGenerator();
console.log(gen.next()); // { value: 1, done: false }
console.log(gen.next()); // { value: 2, done: false }
console.log(gen.next()); // { value: 3, done: false }
console.log(gen.next()); // { value: undefined, done: true }

// Generators are iterable, so they work directly with for...of
for (const num of numberGenerator()) {
  console.log(num); // 1, 2, 3
}
```

Common uses: lazy sequences (infinite series generated on demand), custom iteration logic for objects, and as the foundation older async-flow libraries (like `redux-saga`) were built on.

---

18. Symbols

`Symbol` is a primitive type (ES6) that creates guaranteed-unique, immutable identifiers — mainly used as object keys to avoid naming collisions.

```
const id1 = Symbol("id");
const id2 = Symbol("id"); // different symbol, even with the same description

const user = {};
user[id1] = 101;
user[id2] = 202;

console.log(user[id1]); // 101
console.log(user[id2]); // 202
console.log(id1 === id2); // false
```

Also used to define special object behaviors via well-known symbols, e.g. `Symbol.iterator` to make an object work with `for...of`.

---

19. Currying

Transforms a function taking multiple arguments into a sequence of functions each taking one argument.

```
// Instead of: add(2, 3)
function curryAdd(a) {
  return function (b) {
    return a + b;
  };
}
curryAdd(2)(3); // 5

// Generic curry helper
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) return fn(...args);
    return (...next) => curried(...args, ...next);
  };
}
const add3 = (a, b, c) => a + b + c;
const curriedAdd3 = curry(add3);
curriedAdd3(1)(2)(3);   // 6
curriedAdd3(1, 2)(3);   // 6
curriedAdd3(1, 2, 3);   // 6
```

Used in React/JS for parameterized event handlers, higher-order functions, and reusable callback factories (e.g. `onClick={() => handleDelete(id)}` is a manual curry-like pattern).

---

20. Pure Functions and Side Effects

A **pure function**: (1) always returns the same output for the same input, (2) has no side effects.

```
// Pure
function add(a, b) { return a + b; }
add(2, 3); // always 5

// Impure — modifies external state
let total = 0;
function addToTotal(value) {
  total += value;
  return total;
}
addToTotal(5); // 5
addToTotal(5); // 10 — same input, different output
```

Side effects include: modifying global/outer variables, mutating objects/arrays passed in, API calls, DOM manipulation, `console.log`, updating React state.

Pure functions are preferred because they're predictable, easy to test in isolation, and don't depend on hidden external state.

---

21. Destructuring

Extracts values from arrays or properties from objects into variables in one statement.

```
// Array destructuring — by position
const [first, second] = [10, 20, 30];
const [head, ...rest] = [10, 20, 30, 40]; // head = 10, rest = [20, 30, 40]

// Object destructuring — by property name
const { name, city } = { name: "Sapna", city: "Pune" };

// With renaming and defaults
const { name: userName = "Guest" } = {};
console.log(userName); // "Guest"
```

---

22. Spread vs Rest

Same `...` syntax, opposite purpose: **spread expands, rest collects.**

```
// Spread — expand into individual elements
const nums = [1, 2, 3];
const moreNums = [...nums, 4, 5];           // [1,2,3,4,5]
const merged = { ...userA, ...userB };      // merge objects
console.log(Math.max(...nums));             // spread as function args

// Rest — collect remaining values
function sum(...numbers) {                  // gathers all args into an array
  return numbers.reduce((t, n) => t + n, 0);
}
sum(1, 2, 3, 4); // 10

const { name, ...otherFields } = { name: "Sapna", age: 22, city: "Pune" };
console.log(otherFields); // { age: 22, city: "Pune" }
```

---

23. Garbage Collection

JavaScript automatically allocates memory when objects are created and frees it once they're no longer **reachable** — developers don't manually allocate/free memory (unlike C/C++). The most common algorithm is **mark-and-sweep**: the engine starts from root references (global object, active call stack) and marks everything reachable; anything left unmarked gets collected.

Practical implication: unintentionally-retained references (e.g. via closures, forgotten event listeners, or global caches) prevent collection and cause memory leaks.

---

24. IIFE (Immediately Invoked Function Expression)

A function defined and executed immediately.

```
(function () {
  console.log("Hello");
})();
```

Historically used to create a private scope and avoid polluting the global namespace before ES6 modules and block scoping existed; still occasionally seen in bundler output and some library patterns.

---

25. Modules: CommonJS vs ES Modules

Feature	CommonJS (CJS)	ES Modules (ESM)
Import	`require()`	`import`
Export	`module.exports` / `exports`	`export` / `export default`
Loading	Synchronous	Static, analyzed before execution
Standard	Node.js	Official JS (ES6)
Browser support	No (without bundlers)	Yes (modern browsers)
React/modern JS	Rarely used	Standard

```
// CommonJS
// math.js
module.exports = function add(a, b) { return a + b; };
// app.js
const add = require("./math");

// ES Modules
// math.js
export function add(a, b) { return a + b; }
export default add;
// app.js
import { add } from "./math.js";
```

A file can have multiple named exports but only one default export.

---

26. Array.map(), .filter(), .reduce()

	`map()`	`filter()`	`reduce()`
Purpose	Transform	Select	Combine
Returns	New array, same length	New array, same or smaller	Single accumulated value
Mutates original?	No	No	No

```
const numbers = [1, 2, 3, 4];
numbers.map(n => n * 2);                    // [2, 4, 6, 8]
numbers.filter(n => n % 2 === 0);           // [2, 4]
numbers.reduce((total, n) => total + n, 0); // 10
```

---

27. Array.flat() and Array.flatMap()

```
[1, 2, [3, 4], [5, 6]].flat();          // [1, 2, 3, 4, 5, 6]
[1, 2, 3].flatMap(x => [x * 2]);        // [2, 4, 6] — map + flat(1) in one step

["Hello World", "JS ES6"].flatMap(s => s.split(" "));
// ["Hello", "World", "JS", "ES6"]
```

---

28. Array.sort()

By default, `sort()` converts elements to strings and sorts lexicographically — so numeric sorting needs a compare function.

```
const numbers = [100, 2, 25, 5];
numbers.sort((a, b) => a - b); // [2, 5, 25, 100] — ascending
numbers.sort((a, b) => b - a); // [100, 25, 5, 2] — descending
```

Return value	Meaning
Negative	`a` comes before `b`
Positive	`b` comes before `a`
`0`	Keep original order

`sort()` **mutates** the original array. To avoid mutation: `const sorted = [...arr].sort((a, b) => a - b);`

---

29. Debouncing and Throttling

Both control how often a function runs in response to high-frequency events (typing, scrolling, resizing).

Debounce — wait until the event stops firing, then run once.

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
const debouncedSearch = debounce(search, 500);
input.addEventListener("input", e => debouncedSearch(e.target.value));
```

Throttle — run at most once per fixed interval, no matter how often the event fires.

```
function throttle(fn, delay) {
  let lastTime = 0;
  return function (...args) {
    const now = Date.now();
    if (now - lastTime >= delay) {
      lastTime = now;
      fn(...args);
    }
  };
}
window.addEventListener("scroll", throttle(handleScroll, 1000));
```

Debounce	Throttle
Executes after events stop	Executes at a fixed rate
Resets the timer every event	Ignores events until interval passes
Best for search/autocomplete	Best for scroll/resize/mousemove

Memory trick: Debounce = “wait until they stop.” Throttle = “run at a fixed rate.”

---

30. localStorage, sessionStorage, Cookies

	localStorage	sessionStorage	Cookies
Limit	~5–10 MB	~5 MB	~4 KB
Expiration	Never (until cleared)	Tab/window close	Can set expiry
Sent to server	No	No	Yes, with every matching request
Scope	All tabs, same origin	Current tab only	Available per domain/path until expiry
Best for	Preferences, theme	Multi-step forms, temp data	Auth/session IDs, tracking

```
localStorage.setItem("theme", "dark");
sessionStorage.setItem("step", "3");
document.cookie = "theme=dark; max-age=3600";
```

Security: don't store JWTs or sensitive data in `localStorage` / `sessionStorage` — both are readable by any injected script (XSS risk). Prefer cookies with `HttpOnly` (blocks JS access), `Secure` (HTTPS only), and `SameSite` (mitigates CSRF) for authentication.

---

31. Prototypes & Prototypal Inheritance

A prototype is an object that another object inherits properties/methods from. When a property isn't found directly on an object, JS looks up the **prototype chain** until it finds it or reaches `null`.

```
function Person(name) { this.name = name; }
Person.prototype.greet = function () { console.log(`Hi, ${this.name}`); };

const p = new Person("Sapna");
p.greet(); // "Hi, Sapna" — found via the prototype chain, not on p itself
```

This is memory-efficient: shared methods live once on the prototype rather than being duplicated per instance. ES6 `class` syntax is syntactic sugar over this same mechanism.

Term	Meaning
Prototype	Object other objects inherit from
Prototype chain	Object → its prototype → higher prototypes → `null`
`prototype`	Property on constructor functions holding shared methods
`__proto__`	Legacy way to inspect an object's prototype reference

---

32. Optional Chaining & Nullish Coalescing *(added)*

```
const user = { profile: { name: "Sapna" } };

// Optional chaining — safely access deeply nested properties
console.log(user.profile?.name);     // "Sapna"
console.log(user.address?.city);     // undefined, no error thrown

// Also works on function calls and array access
user.greet?.();          // does nothing if greet doesn't exist, instead of throwing
arr?.[0];

// Nullish coalescing — fallback ONLY for null/undefined (unlike ||, which
// also falls back on 0, "", false)
const count = 0;
console.log(count || 10);  // 10 — wrong if 0 is a valid value!
console.log(count ?? 10);  // 0  — correct, only null/undefined trigger the fallback
```

---

33. Map, Set, WeakMap, WeakSet *(added)*

```
// Map — key-value pairs, keys can be ANY type (unlike plain objects, string-only keys)
const map = new Map();
map.set("name", "Sapna");
map.set(42, "numeric key");
console.log(map.get("name")); // "Sapna"
console.log(map.size);        // 2

// Set — collection of unique values
const set = new Set([1, 2, 2, 3]);
console.log([...set]); // [1, 2, 3] — duplicates removed

// WeakMap/WeakSet — keys must be objects, held WEAKLY (don't prevent garbage
// collection), and are not iterable. Useful for attaching private metadata to
// objects without causing memory leaks.
const wm = new WeakMap();
let obj = {};
wm.set(obj, "metadata");
obj = null; // the WeakMap entry can now be garbage collected
```

	Map	Object	Set	Array
Key/value type	Any type	String/Symbol keys only	Unique values only	Indexed, duplicates allowed
Iteration order	Insertion order	Not guaranteed pre-ES2015	Insertion order	Index order
Size	`.size`	`Object.keys(obj).length`	`.size`	`.length`

34. Error Handling *(added)*

```
try {
  JSON.parse("{ invalid json");
} catch (error) {
  console.error("Parsing failed:", error.message);
} finally {
  console.log("Runs regardless of success or failure");
}

// Custom error types
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = "ValidationError";
  }
}

function validateAge(age) {
  if (age < 0) throw new ValidationError("Age cannot be negative");
  return age;
}

try {
  validateAge(-5);
} catch (err) {
  if (err instanceof ValidationError) {
    console.log("Validation issue:", err.message);
  } else {
    throw err; // re-throw anything unexpected
  }
}
```

In `async` functions, wrap `await` calls in `try/catch` (see §9) rather than chaining `.catch()`, for more linear, readable error handling.

35. Strict Mode *(added)*

`"use strict"` opts into a restricted variant of JS that catches common mistakes: throws on assigning to undeclared variables, disallows duplicate parameter names, makes `this` `undefined` in plain function calls instead of defaulting to the global object, and disallows some legacy syntax.

```
"use strict";
x = 10; // ReferenceError — x was never declared
```

ES6 modules and classes are strict mode by default, so most modern React/Node code runs in strict mode automatically without needing the directive.

---

36. Immutability & Object.freeze() *(added)*

```
const config = Object.freeze({ apiUrl: "https://api.example.com" });
config.apiUrl = "https://evil.com"; // silently fails (throws in strict mode)
console.log(config.apiUrl); // unchanged

// Object.freeze is SHALLOW — nested objects are still mutable
const state = Object.freeze({ user: { name: "Sapna" } });
state.user.name = "Changed"; // this works! freeze didn't reach the nested object
```

For deep immutability, freeze recursively or use a library (Immer, Immutable.js). This matters directly for React/Redux, where state should be treated as immutable and updated by creating new objects, not mutating existing ones.

---

37. Event Delegation, Bubbling & Capturing *(added)*

DOM events propagate in two phases: **capturing** (root → target) then **bubbling** (target → root). Event delegation exploits bubbling by attaching a single listener to a parent instead of one per child.

```
// Instead of adding a click listener to every <li>...
document.getElementById("list").addEventListener("click", (e) => {
  if (e.target.tagName === "LI") {
    console.log("Clicked item:", e.target.textContent);
  }
});
```

Benefits: fewer listeners (better memory/performance), automatically works for dynamically added children. Use `e.stopPropagation()` to stop bubbling, and `{ capture: true }` in `addEventListener` to hook the capture phase instead.

---

38. What Actually Happens with `new` *(added)*

```
function Person(name) {
  this.name = name;
}

const p = new Person("Sapna");
```

`new Person("Sapna")` does four things: 1. Creates a brand-new empty object 2. Sets that object's internal prototype to `Person.prototype` 3. Calls `Person` with `this` bound to the new object 4. Returns the new object (unless the constructor explicitly returns a different object)

---

39. ES6 Features Summary

- **Block scoping** — `let` / `const` instead of `var`
- **Arrow functions** — concise syntax, lexical `this`
- **Template literals** — `` `Hello ${name}` `` for interpolation and multi-line strings
- **Destructuring & defaults** — unpack arrays/objects, default parameter values
- **Rest & spread** — `...` for variadic args and expanding/merging
- **Modules & classes** — `import` / `export`, `class` syntax over prototypes
- **Promises** — cleaner async flow than callbacks
- **New collections** — `Map`, `Set` (and their `Weak` variants)
- `for...of` — iterate over iterable values directly
- **Symbols** — unique property keys

---

40. Performance: Optimizing a Large Fetch (Interview Scenario)

Q: Fetching 1000 entries is slow — what would you do?

1. **Paginate / infinite scroll** — don't fetch everything upfront; load in pages (`?page=1&limit=20`) — highest leverage, cuts both payload and render cost
2. **Virtualize the list** — `react-window` / `react-virtualized` render only visible rows, not all 1000 DOM nodes
3. **Reduce payload** — return only needed fields, enable gzip/Brotli compression
4. **Cache** — React Query/SWR/RTK Query to avoid redundant re-fetches; leverage `ETag` / `Cache-Control` headers
5. **Debounce input-triggered fetches** — avoid firing a request per keystroke
6. **Parallelize independent requests** — `Promise.all` instead of sequential `await`s
7. **Perceived performance** — skeleton loaders, or stream partial results if the API supports it
8. **Offload heavy client-side processing** — Web Workers for sorting/filtering large datasets so the UI thread doesn't freeze
9. **Backend optimization** — proper DB indexing, or GraphQL if over/under-fetching is recurring

Prioritized answer: pagination (reduce what's fetched) → virtualization (reduce what's rendered) → caching (avoid re-fetching) covers most real cases; mention compression and Web Workers as secondary wins.

Quick Reference — What to Reach For

Problem	Solution
Need a value only if not null/undefined	`??`
Safely access a possibly-missing nested property	`?.`
Unique values, fast lookup	`Set`
Key-value pairs with non-string keys	`Map`
Metadata on an object without leaking memory	`WeakMap`
Repeated expensive computation	Memoization
Search input firing too many requests	Debounce
Scroll/resize handler firing too often	Throttle
One listener for many dynamic children	Event delegation
Avoid mutating shared state	`Object.freeze` / spread copies
Clone nested data safely	`structuredClone`
Multiple async calls, need all to succeed	`Promise.all`
Multiple async calls, want every result regardless	`Promise.allSettled`
Want whichever settles first (e.g. timeout race)	`Promise.race`
Want first success, ignore failures unless all fail	`Promise.any`